LES arguments nommés
Les fonctions variadiques
Déballage de Tableau
Closure arrow Functions
Portée des variables

En PHP, une **fonction** est un bloc de code réutilisable conçu pour exécuter une tâche précise. Elle permet de structurer le code, d'éviter les répétitions et de faciliter la maintenance.
On distingue deux grandes familles de fonctions : les **fonctions natives** (intégrées par défaut dans le langage, comme `strlen()` ou `array_push()`) et les **fonctions utilisateurs** (que vous créez vous-même).

---

1. Déclarer et appeler une fonction de base

Pour déclarer une fonction, on utilise le mot-clé `function`, suivi de son nom (insensible à la casse) et d'une paire de parenthèses.

```
// Déclaration
function saluer() {
    echo "Bonjour tout le monde !";
}

// Appel de la fonction
saluer(); // Affiche : Bonjour tout le monde !
```

2. Les paramètres et arguments

Une fonction peut recevoir des informations pour fonctionner. On définit des **paramètres** lors de la création de la fonction, et on lui passe des **arguments** (les valeurs réelles) lors de l'appel.

```
function direBonjour($nom) {
    echo "Bonjour " . $nom . " !";
}

direBonjour("Alice"); // Affiche : Bonjour Alice !
```

Il est aussi possible de définir une **valeur par défaut** pour un paramètre, ce qui le rend optionnel lors de l'appel.

```
function bienvenue($role = "Visiteur") {
    echo "Bienvenue, votre rôle est : " . $role;
```

```
}

bienvenue();        // Affiche : Bienvenue, votre rôle est : Visiteur
bienvenue("Admin"); // Affiche : Bienvenue, votre rôle est : Admin
```

3. Retourner une valeur (`return`)

Pour qu'une fonction transmette un résultat au reste du script (au lieu de simplement l'afficher), on utilise l'instruction `return`. Dès que `return` est exécuté, la fonction s'arrête.

```
function additionner($a, $b) {
    return $a + $b;
}

$resultat = additionner(5, 3);
echo $resultat; // Affiche : 8
```

4. Le typage (PHP 7 et PHP 8)

Il est fortement recommandé de **typer** les paramètres d'entrée ainsi que la valeur de retour afin de rendre le code plus robuste et plus sûr.

```
function multiplier(float $a, float $b): float {
    return $a * $b;
}
```

5. Les fonctionnalités avancées de PHP 8

Depuis les versions récentes, PHP offre des outils puissants pour manipuler les fonctions :

Les arguments nommés (PHP 8.0) : Ils permettent de passer des valeurs à une fonction en se basant sur le nom du paramètre plutôt que sur sa position.

```
// L'ordre des arguments n'a plus d'importance si on les nomme
bienvenue(role: "Modérateur");
```

Les fonctions anonymes (closures) : Des fonctions sans nom, souvent utilisées comme fonctions de rappel (*callbacks*).

```
$carre = function($nombre) {
    return $nombre * $nombre;
};
echo $carre(4); // Affiche : 16
```

Les fonctions fléchées (Arrow functions) : Une syntaxe plus courte pour les fonctions anonymes à une seule ligne.

```
$double = fn($n) => $n * 2;
```

Les arguments nommés (PHP 8.0)

Introduits avec **PHP 8.0**, les **arguments nommés** (ou *named arguments*) permettent de passer des valeurs à une fonction en spécifiant le nom du paramètre, plutôt que de se baser uniquement sur leur ordre de déclaration.
Voici un guide complet pour comprendre leur fonctionnement, leurs avantages et les règles à respecter.

---

1. Syntaxe de base

Pour utiliser un argument nommé, il suffit de calquer la syntaxe sur le nom du paramètre dans la fonction en écrivant : `nom_du_parametre: valeur`.

```
// Déclaration classique d'une fonction
function creerUtilisateur($nom, $email, $role) {
    echo "Utilisateur $nom ($email) créé avec le rôle $role.";
}

// Appel classique (positionnel) : l'ordre doit être strict
creerUtilisateur("Alice", "alice@example.com", "Admin");

// Appel avec arguments nommés : l'ordre n'a plus d'importance !
creerUtilisateur(role: "Admin", nom: "Alice", email: "alice@example.com");
```

2. Les grands avantages des arguments nommés

A. Ignorer les paramètres optionnels par défaut

C'est l'un des gains de productivité les plus importants. Avant PHP 8, si vous vouliez modifier uniquement le 4ème paramètre d'une fonction, vous deviez réécrire les valeurs par défaut des paramètres 2 et 3.

```
function configurerSession($duree = 3600, $securise = true, $httpOnly = true, $nom = "PHPSESSID") {
    // ...
}

// AVANT PHP 8 : Obligé de réécrire true, true pour changer le nom
configurerSession(3600, true, true, "MA_SESSION");

// AVEC PHP 8 : On saute directement au paramètre qui nous intéresse
configurerSession(nom: "MA_SESSION");
```

B. Amélioration immédiate de la lisibilité

Le code devient auto-documenté. En lisant l'appel de la fonction, on comprend instantanément à quoi correspond chaque valeur sans avoir à ouvrir le fichier source de la fonction.

```
// Difficile à décoder sans contexte :
setcookie("mon_cookie", "valeur", 3600, "/", "", true, true);

// Beaucoup plus clair :
setcookie(name: "mon_cookie", value: "valeur", secure: true, httponly: true);
```

3. Les règles d'utilisation et contraintes

- **Mixer les styles :** Vous pouvez mélanger les arguments positionnels (classiques) et nommés, mais les **arguments positionnels doivent toujours être placés en premier**.
- `// VALIDE : positionnel d'abord, puis nommés`
- `creerUtilisateur("Alice", role: "Admin", email: "alice@example.com");`
-
- `// ERREUR (Fatal Error) : impossible de mettre un positionnel après un nommé`
- `creerUtilisateur(nom: "Alice", "alice@example.com", role: "Admin");`
- **Pas de doublons :** Vous ne pouvez pas passer une valeur pour un même paramètre à la fois de manière positionnelle et de manière nommée.
- `// ERREUR : $nom reçoit "Alice" (position 1) ET "Bob" (nommé)`
- `creerUtilisateur("Alice", "alice@example.com", "Admin", nom: "Bob");`
- **Héritage et renommage :** Si une classe enfant redéfinit une méthode en changeant le nom d'un paramètre, c'est le nom du paramètre de la classe de l'objet exécuté au moment de l'appel qui compte. Il est donc recommandé de garder les mêmes noms de paramètres lors de l'héritage pour éviter les mauvaises surprises.

4. Utilisation avec le déballage de tableaux (Splat Operator)

Depuis PHP 8.1, vous pouvez déballer un tableau associatif directement dans une fonction. Les clés du tableau feront office d'arguments nommés.

```
$donnees = [
    'email' => 'contact@test.com',
    'role'  => 'Auteur',
```

```
    'nom'   => 'Jean'
];

// Les clés correspondent exactement aux paramètres de la fonction
creerUtilisateur(...$donnees);
```

Le **déballage de tableau** (connu sous le nom de *Array Unpacking* ou *Splat Operator* `...`) permet d'extraire les éléments d'un tableau pour les injecter directement soit **dans une fonction** (comme des arguments), soit **dans un autre tableau**.
Introduite pour les tableaux simples en PHP 7.4, cette fonctionnalité a été étendue aux **tableaux associatifs dans PHP 8.1**.

---

1. Déballage dans les arguments d'une fonction

Au lieu de passer les éléments d'un tableau un par un, le symbole `...` "ouvre" le tableau pour distribuer ses valeurs aux paramètres correspondants.

A. Avec un tableau indexé (par position)

Les éléments du tableau sont injectés dans l'ordre de leurs index (0, 1, 2...).

```
function calculerVolume(int $longueur, int $largeur, int $hauteur): int {
    return $longueur * $largeur * $hauteur;
}

$dimensions = array(10, 5, 2);

// Déballage du tableau : PHP lit cela comme calculerVolume(10, 5, 2)
$volume = calculerVolume(...$dimensions);

echo $volume; // Affiche : 100
```

B. Avec un tableau associatif (par arguments nommés) — *Depuis PHP 8.1*

Les **clés du tableau doivent correspondre exactement** aux noms des paramètres de la fonction. L'ordre des clés dans le tableau n'a plus d'importance.

```
function creerProfil(string $pseudo, string $statut, bool $estActif) {
    echo "Profil : $pseudo ($statut) - Actif : " . ($estActif ? 'Oui' :
'Non');
}
```

```
$donneesFormulaire = [
    'statut'   => 'Modérateur',
    'estActif' => true,
    'pseudo'   => 'Shadow99', // L'ordre diffère de la fonction, mais les
clés correspondent
];

// PHP associe automatiquement les clés aux paramètres nommés
creerProfil(...$donneesFormulaire);
// Affiche : Profil : Shadow99 (Modérateur) - Actif : Oui
```

---

2. Déballage pour fusionner des tableaux

Le déballage est une alternative moderne, plus rapide et plus lisible à la fonction classique `array_merge()`.

A. Fusion de tableaux indexés (*Depuis PHP 7.4*)

Vous pouvez fusionner et insérer des éléments n'importe où lors de la création d'un nouveau tableau.

```
$legumes = ['Carotte', 'Poireau'];
$fruits  = ['Pomme', 'Banane'];

// Fusion simple avec ajout d'un élément au milieu
$panier = ['Pain', ...$legumes, 'Fromage', ...$fruits];

print_r($panier);
/*
Affiche :
Array (
    [0] => Pain
    [1] => Carotte
    [2] => Poireau
    [3] => Fromage
    [4] => Pomme
    [5] => Banane
)
*/
```

B. Fusion de tableaux associatifs (*Depuis PHP 8.1*)

Si deux tableaux possèdent **la même clé textuelle**, la règle de PHP est que **la dernière valeur écrase la précédente**.

```
$configurationParDefaut = [
    'theme' => 'clair',
    'langue' => 'fr',
    'notifications' => true,
];

$preferencesUtilisateur = [
    'theme' => 'sombre', // Va écraser 'clair'
    'langue' => 'en',   // Va écraser 'fr'
```

```
];

// Fusion des deux configurations
$configurationFinale = [
    ...$configurationParDefaut,
    ...$preferencesUtilisateur,
    'version' => '2.0' // On peut ajouter de nouvelles clés à la volée
];

print_r($configurationFinale);
/*
Affiche :
Array (
    [theme] => sombre
    [langue] => en
    [notifications] => 1
    [version] => 2.0
)
*/
```

⚠️ Piège fréquent à éviter

Si vous utilisez le déballage de tableau associatif dans une fonction (PHP 8.1+), votre tableau **ne doit pas contenir de clés numériques** mélangées à des clés textuelles, sous peine de déclencher une erreur critique (*Fatal Error*).

```
$mauvaisTableau = [
    'pseudo' => 'Luc',
    0 => 'Valeur parasite' // Clé numérique interdite ici
];

// Génère une Error: Cannot combine positional and named arguments
creerProfil(...$mauvaisTableau);
```

Les **fonctions variadiques** sont l'exact inverse du déballage de tableau. Au lieu d'ouvrir un tableau pour alimenter une fonction, elles permettent à une fonction d'accepter un **nombre inconnu ou variable d'arguments** et de les regrouper automatiquement dans un unique tableau.

Pour les déclarer, on utilise le même symbole de trois points `...` (appelé opérateur *splat* ou *rest*) juste avant le nom du paramètre.

---

1. Exemple de base : Une fonction de calcul

Imaginons une fonction qui doit additionner une liste de nombres, peu importe qu'il y en ait 2, 5 ou 10.

```
// Le paramètre ...$nombres va capturer TOUS les arguments reçus
function additionnerTout(...$nombres) {
    $total = 0;

    // $nombres est traité en interne comme un tableau classique
    foreach ($nombres as $nombre) {
        $total += $nombre;
    }

    return $total;
}

// Appels avec un nombre variable d'arguments
echo additionnerTout(5, 10);        // Affiche : 15
echo additionnerTout(1, 2, 3, 4, 5); // Affiche : 15
```

2. Combiner des paramètres fixes et un paramètre variadique

Vous pouvez tout à fait exiger un ou plusieurs arguments obligatoires au début, et laisser le reste en variadique.

⚠ **Règle absolue :** Le paramètre variadique précédé de `...` doit obligatoirement être le **tout dernier paramètre** de la fonction.

```
function introduireEquipe($nomDuChef, ...$membres) {
    echo "Le chef de projet est : " . $nomDuChef . "<br>";
    echo "Les membres de l'équipe sont : " . implode(", ", $membres);
}

// "Alice" va dans $nomDuChef. Tous les autres vont dans le tableau $membres.
introduireEquipe("Alice", "Bob", "Charlie", "David");
/*
Affiche :
Le chef de projet est : Alice
Les membres de l'équipe sont : Bob, Charlie, David
*/
```

3. Ajouter du typage pour plus de sécurité

Vous pouvez imposer un type aux éléments capturés par l'opérateur variadique. Si un type est spécifié, **chaque argument** envoyé à la fonction devra obligatoirement respecter ce type.

```
// Tous les arguments fournis devront être des chaînes de caractères (string)
function listerMotsCles(string ...$mots) {
    print_r($mots);
}

// Fonctionne parfaitement
listerMotsCles("PHP", "Programmation", "Web");

// Déclenchera une erreur (TypeError) car 2026 n'est pas une string
listerMotsCles("PHP", 2026);
```

4. Le combo parfait : Variadique + Déballage

Ces deux fonctionnalités s'associent à la perfection. Vous pouvez capturer des arguments de manière variadique dans une fonction, puis les déballer pour les envoyer vers une autre fonction.

```
function logevenement($type, ...$details) {
    // On déballe $details pour passer ses éléments à une autre fonction
    sauvegarderEnBaseDeDonnees($type, ...$details);
}
```

Voici les deux concepts essentiels concernant la gestion des données et la mémoire dans les fonctions PHP.

La portée des variables (*Variable Scope*)

En PHP, une variable déclarée à l'extérieur d'une fonction n'est pas automatiquement accessible à l'intérieur de celle-ci, et inversement. C'est ce qu'on appelle la **portée**.

A. Les variables locales

Toute variable créée à l'intérieur d'une fonction est **locale**. Elle naît quand la fonction s'exécute et elle est **détruite** dès que la fonction se termine.

```
function testerPortee() {
    $texte = "Je suis locale";
}

testerPortee();
echo $texte; // Erreur ! PHP ne connaît pas $texte en dehors de la fonction.
```

B. Les variables globales

Si vous avez besoin d'utiliser une variable du script principal à l'intérieur d'une fonction, vous devez explicitement utiliser le mot-clé `global`.

```
$pays = "France"; // Variable globale

function afficherPays() {
    global $pays; // On indique à PHP qu'on utilise la variable externe
    echo "Le pays est : " . $pays;
}

afficherPays(); // Affiche : Le pays est : France
```

Note : L'utilisation abusive de `global` est souvent déconseillée, car elle rend le code difficile à maintenir. Il est préférable de passer les variables en arguments.

C. Les variables statiques (`static`)

Une variable locale oublie sa valeur dès que la fonction se termine. Si vous voulez qu'une variable locale **garde sa valeur en mémoire** entre plusieurs appels successifs de la fonction, on utilise le mot-clé `static`.

```
function incrementerCompteur() {
    static $compteur = 0; // Initialisé seulement au PREMIER appel
    $compteur++;
    echo "Valeur : " . $compteur . "<br>";
}

incrementerCompteur(); // Affiche : Valeur : 1
incrementerCompteur(); // Affiche : Valeur : 2
incrementerCompteur(); // Affiche : Valeur : 3
```

---

2. Le passage d'arguments par référence (`&`)

Par défaut, en PHP, les arguments sont passés **par valeur**. Cela signifie que PHP crée une **copie** de la variable originale dans la fonction. Si vous modifiez la variable à l'intérieur de la fonction, la variable d'origine reste intacte.
Le passage **par référence** (en ajoutant le symbole `&` devant le paramètre) permet de lier la variable de la fonction directement à la variable d'origine. Modifier la variable locale **modifie directement la variable externe**.

Exemple comparatif :

```
// 1. Passage par VALEUR (comportement par défaut)
function ajouterDixParValeur($nombre) {
    $nombre += 10;
}

$maValeur = 5;
ajouterDixParValeur($maValeur);
echo $maValeur; // Affiche : 5 (la variable d'origine n'a pas bougé)


// 2. Passage par RÉFÉRENCE (avec le &)
function ajouterDixParReference(&$nombre) {
    $nombre += 10;
}

$maValeur = 5;
ajouterDixParReference($maValeur);
echo $maValeur; // Affiche : 15 (la variable d'origine a été modifiée !)
```

En PHP, les **fonctions anonymes** (ou *closures*) et les **fonctions fléchées** (*arrow functions*) permettent de créer des fonctions "à la volée", sans leur attribuer de nom. Elles sont omniprésentes dans le PHP moderne (PHP 7 et PHP 8), notamment pour être passées comme arguments à des fonctions natives (comme `array_map` ou `usort`).

Voici une analyse détaillée de ces deux outils, de leurs différences et de leurs cas d'usage.

Les Fonctions Anonymes (*Closures*)

Introduites en PHP 5.3, les fonctions anonymes sont des fonctions sans nom qui peuvent être stockées dans des variables ou passées en paramètres. Elles se comportent techniquement comme des instances de la classe interne `Closure`.

Syntaxe de base

```
// Stockage dans une variable
$saluer = function(string $nom) {
    return "Bonjour " . $nom;
};

// Appel via la variable
echo $saluer("Alice"); // Affiche : Bonjour Alice
```

L'héritage des variables parentes (`use`)

Par défaut, une fonction anonyme n'a pas accès aux variables définies à l'extérieur de sa portée locale. Pour lui donner accès à une variable du script parent, il faut utiliser le mot-clé **`use`**.

```
$tva = 0.20;

// On "injecte" la variable $tva dans la portée de la fonction
$calculerTTC = function(float $prixHT) use ($tva) {
    return $prixHT * (1 + $tva);
};
```

```
echo $calculerTTC(100); // Affiche : 120
```
*Note : Les variables transmises via `use` sont passées **par valeur** (une copie est faite au moment de la définition de la fonction). Si vous souhaitez modifier la variable d'origine depuis l'intérieur, il faut ajouter un symbole `&` (ex: `use (&$tva)`).*

Les Fonctions Fléchées (*Arrow Functions*)

Introduites en **PHP 7.4**, les fonctions fléchées apportent une syntaxe beaucoup plus concise pour les fonctions anonymes à **courte portée** (qui ne contiennent qu'une seule expression).

Syntaxe de base

Elles utilisent le mot-clé `fn`, suivi des paramètres, d'une flèche `=>`, et de l'expression à retourner. **L'instruction `return` est implicite.**

```
// Syntaxe ultra-courte
$carre = fn(int $n) => $n * $n;

echo $carre(4); // Affiche : 16
```

La capture automatique des variables (La grande différence !)

Contrairement aux fonctions anonymes classiques, les fonctions fléchées **capturent automatiquement** toutes les variables de la portée parente dont elles ont besoin. Le mot-clé `use` n'est plus nécessaire.

```
$facteur = 3;

// $facteur est capturé automatiquement par valeur
$multiplier = fn(int $nombre) => $nombre * $facteur;

echo $multiplier(5); // Affiche : 15
```
*Attention : La capture se fait toujours **par valeur**. Vous ne pouvez pas modifier directement une variable externe depuis une fonction fléchée.*

---

3. Tableau comparatif : Anonyme vs Fléchée

Caractéristique	**Fonction Anonyme (`function`)**	**Fonction Fléchée (`fn`)**
Longueur du code	Idéal pour les blocs de code complexes multi-lignes.	Idéal pour les expressions courtes sur une seule ligne.
Accès aux variables parentes	Manuel, via le mot-clé `use (...)`.	Automatique (par valeur), pas de `use`.

Retour de valeur	Requiert explicitement le mot-clé `return`.	Implicite (la valeur après `=>` est retournée).
Modification des variables parentes	Possible si passées par référence (`use (&$v)`).	Impossible (copie par valeur obligatoire).

---

4. Exemples concrets d'utilisation

Ces fonctions brillent particulièrement lorsqu'elles sont combinées avec les fonctions natives de manipulation de tableaux de PHP.

Exemple 1 : Filtrer un tableau (`array_filter`)

Ici, la fonction fléchée excelle par sa concision pour extraire uniquement les nombres pairs :

```
$nombres = array(1, 2, 3, 4, 5, 6);

// Utilisation d'une fonction fléchée
$pairs = array_filter($nombres, fn($n) => $n % 2 === 0);

print_r($pairs); // Contient array(2, 4, 6)
```

Exemple 2 : Formater des données (`array_map`)

Imaginons qu'on veuille appliquer un formatage de texte à tout un tableau :

```
$utilisateurs = array("alice", "bob", "charlie");

// Met la première lettre en majuscule
$utilisateursFormates = array_map(fn($nom) => ucfirst($nom), $utilisateurs);

print_r($utilisateursFormates); // Contient array("Alice", "Bob", "Charlie")
```

Exemple 3 : Tri personnalisé complexe (`usort`)

Si le traitement nécessite plusieurs lignes ou des vérifications complexes, la fonction anonyme classique reste le meilleur choix :

```
$produits = array(
    array('nom' => 'Livre', 'prix' => 15),
    array('nom' => 'Stylo', 'prix' => 3),
    array('nom' => 'Sac', 'prix' => 45)
);

// Tri par prix croissant avec une fonction anonyme classique
usort($produits, function($a, $b) {
    if ($a['prix'] === $b['prix']) {
        return 0;
    }
    return ($a['prix'] < $b['prix']) ? -1 : 1;
```

```
});
```

La fonction `usort()` est une fonction native de PHP très puissante qui permet de trier un tableau en utilisant un **algorithme de tri personnalisé**. Le "u" de `usort` signifie d'ailleurs *user-defined* (défini par l'utilisateur).
Pour comprendre notre exemple, il faut comprendre la logique de cette fonction.

---

1. La règle d'or de `usort()`

Pour trier un tableau complexe (comme notre tableau de produits), `usort()` va parcourir le tableau et comparer les éléments deux par deux (que l'on appelle souvent `$a` et `$b`).
Puisqu'il s'agit de tableaux imbriqués, PHP ne sait pas par défaut s'il doit trier par le *nom* ou par le *prix*. C'est **votre fonction anonyme** qui va lui dicter la règle. Votre fonction doit obligatoirement retourner l'un de ces trois résultats après avoir comparé `$a` et `$b` :

- `0` : Si les deux éléments sont **égaux**. Leur ordre ne change pas.
- **Un nombre négatif (`-1`)** : Si `$a` est **plus petit** que `$b`. Cela indique à PHP que `$a` doit être placé *avant* `$b`.
- **Un nombre positif (`1`)** : Si `$a` est **plus grand** que `$b`. Cela indique à PHP que `$a` doit être placé *après* `$b`.

---

2. L'explication ligne par ligne du code

Reprenons le code :

```
$produits = array(
    array('nom' => 'Livre', 'prix' => 15),
    array('nom' => 'Stylo', 'prix' => 3),
    array('nom' => 'Sac', 'prix' => 45)
);

// Applique le tri personnalisé sur le tableau $produits
usort($produits, function($a, $b) {

    // Étape 1 : Si les prix sont identiques, on ne change rien
    if ($a['prix'] === $b['prix']) {
        return 0;
    }

    // Étape 2 : Si le prix de $a est inférieur à $b, on renvoie -1 (place $a
avant)
    // Sinon, on renvoie 1 (place $a après)
    return ($a['prix'] < $b['prix']) ? -1 : 1;
});
```

Ce que fait PHP en arrière-plan (Exemple pas à pas) :

1. PHP prend deux éléments au hasard pour démarrer, par exemple **Livre ($a)** et **Stylo ($b)**.
2. Il extrait leurs prix : `$a['prix']` vaut `15`, et `$b['prix']` vaut `3`.
3. Il vérifie l'étape 1 : `15 === 3` ? Faux.
4. Il passe à l'étape 2 : Est-ce que `15 < 3` ? Faux. Le code passe donc après les deux points `:` et renvoie `1`.
5. Puisque le résultat est `1`, PHP comprend que le **Stylo** (3) doit passer *avant* le **Livre** (15).
6. PHP répète l'opération avec les autres éléments jusqu'à ce que tout le tableau soit parfaitement ordonné du plus petit au plus grand prix.

---

3. La version moderne de PHP 7 & 8 (L'opérateur Vaisseau Spatial `<=>`)

Pour vous simplifier la vie, PHP a introduit l'opérateur de comparaison combinée `<=>` (surnommé le *Spaceship operator*). Cet opérateur fait exactement la même chose que notre bloc de code `if / else` : il renvoie automatiquement `-1`, `0` ou `1`.

On peut donc réécrire l'exemple de manière beaucoup plus courte avec une **fonction fléchée** :

```
// Version moderne, ultra-courte et identique
usort($produits, fn($a, $b) => $a['prix'] <=> $b['prix']);
```

Si vous vouliez trier par prix **décroissant** (du plus cher au moins cher), il suffirait d'inverser `$a` et `$b` :

```
usort($produits, fn($a, $b) => $b['prix'] <=> $a['prix']);
```

Voici une série d'exercices pratiques classés par notion pour valider vos acquis en PHP. Chaque exercice comprend un énoncé, un exemple d'attendu et sa solution détaillée.

Exercice 1 : Fonctions de base et Passage de paramètres (par valeur et référence)

Énoncé :

1. Créez une fonction nommée `calculerRemise` qui prend en paramètre un prix HT (float) et un pourcentage de remise (int). Elle doit retourner le montant de la remise. Le paramètre de la remise doit avoir une valeur par défaut de **10** (%).

2. Créez une seconde fonction nommée `appliquerTaxe` qui prend en paramètre un prix (float) **par référence** et un taux de taxe (float). Cette fonction ne retourne rien, mais doit modifier directement la variable d'origine pour lui ajouter la taxe.

Solution :

```
// 1. Fonction avec paramètre optionnel
function calculerRemise(float $prixHT, int $pourcentage = 10): float {
    return $prixHT * ($pourcentage / 100);
}

// 2. Fonction avec passage par référence (&)
function appliquerTaxe(float &$prix, float $tauxTaxe = 0.20): void {
    $prix = $prix * (1 + $tauxTaxe);
}

// --- Test des fonctions ---
$prixArticle = 100.0;
$remise = calculerRemise($prixArticle); // Utilise les 10% par défaut
echo "Montant remise : " . $remise . "€<br>"; // Affiche : 10€

$prixNet = $prixArticle - $remise; // 90.0

appliquerTaxe($prixNet); // Modifie directement $prixNet en lui ajoutant 20%
echo "Prix final TTC : " . $prixNet . "€"; // Affiche : 108€
```

Exercice 2 : Arguments nommés

Énoncé :
Vous disposez de la fonction suivante qui génère un code HTML pour une image :

```
function genererImage(string $src, string $alt = "Image", int $largeur = 300,
int $hauteur = 200): string {
    return "<img src='$src' alt='$alt' width='$largeur' height='$hauteur'>";
}
```

En utilisant les **arguments nommés** (PHP 8+), écrivez l'appel de fonction permettant de modifier uniquement la `hauteur` à **500** et la `source (src)` à **"logo.png"**, sans réécrire les valeurs par défaut de `alt` et `largeur`.

Solution :

```
function genererImage(string $src, string $alt = "Image", int $largeur = 300,
int $hauteur = 200): string {
    return "<img src='$src' alt='$alt' width='$largeur' height='$hauteur'>";
}

// Appel avec arguments nommés : on saute les paramètres optionnels inutiles
$html = genererImage(hauteur: 500, src: "logo.png");

echo htmlentities($html);
// Affiche : <img src='logo.png' alt='Image' width='300' height='500'>
```

Exercice 3 : Déballage de tableau (*Array Unpacking*)

Énoncé :

1. Créez une fonction `creerUrl` qui accepte trois paramètres : `$protocole`, `$domaine` et `$page`.
2. Vous disposez d'un tableau associatif contenant ces informations. Utilisez le **déballage de tableau** pour alimenter votre fonction. *(Rappel : utilisez la syntaxe `array(...)` pour l'exercice si vous craignez les bugs d'affichage).*

Solution :

```
function creerUrl(string $protocole, string $domaine, string $page): string {
    return $protocole . "://" . $domaine . "/" . $page;
}

// Tableau associatif dont les clés correspondent aux paramètres de la
fonction
$configuration = array(
    'page' => 'contact.php',
    'protocole' => 'https',
    'domaine' => 'mon-site.com'
);

// Déballage du tableau associatif (PHP 8.1+)
$url = creerUrl(...$configuration);

echo $url; // Affiche : https://mon-site.com
```

Exercice 4 : Fonctions anonymes (avec `use`)

Énoncé :
Vous avez un tableau de prix HT. Vous devez créer une fonction anonyme stockée dans une variable `$convertisseur` qui applique un coefficient de conversion monétaire (ex: taux de change Dollar/Euro). Ce taux de change est défini dans une variable globale externe. Utilisez `array_map` et le mot-clé `use` pour obtenir un nouveau tableau de prix convertis.

Solution :

```
$prixHT = array(10, 20, 50, 100);
$tauxDollar = 1.10; // Variable dans la portée parente

// Fonction anonyme qui importe $tauxDollar de l'extérieur
$convertisseur = function(float $prix) use ($tauxDollar): float {
    return $prix * $tauxDollar;
};

// Application de la fonction anonyme à tout le tableau
$prixEnDollars = array_map($convertisseur, $prixHT);

print_r($prixEnDollars);
/*
Affiche :
Array ( [0] => 11 [1] => 22 [2] => 55 [3] => 110 )
*/
```

Exercice 5 : Fonctions fléchées (*Arrow functions*)

Énoncé :
Reprenez l'exercice 4 (le convertisseur de monnaie), mais réécrivez-le intégralement en utilisant une **fonction fléchée** (`fn`) afin de rendre le code beaucoup plus court et de profiter de la capture automatique de variable.
Solution :

```
$prixHT = array(10, 20, 50, 100);
$tauxDollar = 1.10;

// Version ultra-courte avec une fonction fléchée
// Pas besoin de "use", $tauxDollar est capturé automatiquement !
$prixEnDollars = array_map(fn($prix) => $prix * $tauxDollar, $prixHT);

print_r($prixEnDollars);
/*
Affiche exactement le même résultat :
Array ( [0] => 11 [1] => 22 [2] => 55 [3] => 110 )
*/
```

En PHP, une **méthode de rappel** (*callback*) est simplement une fonction que vous passez en argument à une autre fonction, pour que cette dernière puisse l'exécuter ("rappeler") au moment opportun.
Le type natif utilisé pour représenter une fonction de rappel en PHP est **`callable`**.
Voici comment créer vos propres fonctions qui acceptent des *callbacks*, en utilisant toutes les syntaxes modernes que nous avons étudiées.

---

1. Créer une fonction qui accepte un `callable`

Pour accepter une fonction de rappel, il suffit de typer un paramètre avec le mot-clé `callable`.
Pour exécuter ce callback à l'intérieur de votre fonction, vous ajoutez des parenthèses à la variable, comme pour une fonction classique : `$callback()`.

```
// Notre fonction principale acceptant un callback
function verifierTexte(string $texte, callable $formatteur) {
    // On exécute le callback sur le texte reçu
    $texteModifie = $formatteur($texte);

    echo "Résultat final : " . $texteModifie;
}
```

---

2. Les 4 façons de passer un callback à cette fonction

Une fois votre fonction programmée, vous pouvez lui envoyer un callback sous différentes formes selon votre besoin.

A. Avec une fonction fléchée (Le plus rapide)

Idéal si le traitement tient sur une seule ligne. Elle capture automatiquement les variables parentes si besoin.

```
// On passe directement la fonction fléchée en argument
verifierTexte("sécurité web", fn($t) => strtoupper($t));

// Affiche : Résultat final : SÉCURITÉ WEB
```

B. Avec une fonction anonyme classique (`function () use`)

Idéal si le traitement nécessite plusieurs lignes ou une logique complexe (conditions, boucles).

```
$prefixe = "[ALERTE]";

verifierTexte("Le serveur est en surchauffe !", function($message) use
($prefixe) {
    $messageNettoye = trim($message);
    return $prefixe . " " . $messageNettoye;
});

// Affiche : Résultat final : [ALERTE] Le serveur est en surchauffe !
```

C. En passant le nom d'une fonction standard (sous forme de chaîne)

Si vous possédez déjà une fonction classique nommée dans votre code (ou une fonction native de PHP comme `ucfirst` ou `md5`), vous pouvez simplement passer **son nom sous forme de chaîne de caractères (`string`)**. PHP comprendra qu'il doit chercher la fonction portant ce nom.

```
// Utilisation d'une fonction native PHP comme callback
verifierTexte("bonjour", "ucfirst");

// Affiche : Résultat final : Bonjour
```

D. Depuis PHP 8.1 : La syntaxe de première classe (`maFonction(...)`)

Pour éviter d'utiliser des chaînes de caractères (qui ne sont pas pratiques pour l'autocomplétion dans les éditeurs de code), PHP 8.1 a introduit la syntaxe avec les trois points `...`. Elle transforme n'est plus ni moins n'importe quelle fonction existante en un objet `Closure` manipulable.

```
function inverserTexte(string $str): string {
    return strrev($str);
}

// Syntaxe moderne de PHP 8.1 pour désigner la fonction comme un callable
verifierTexte("PHP 8", inverserTexte(...));
```

```
// Affiche : Résultat final : 8 PHP
```

💡 Un cas concret : Un système de filtre personnalisé

Imaginons que vous développiez un système de panier d'achat et que vous vouliez filtrer des produits selon un critère que l'utilisateur choisit dynamiquement (par exemple : "uniquement les produits chers", ou "uniquement les produits en stock").

```
$panier = array(
    array('nom' => 'Clavier', 'prix' => 80),
    array('nom' => 'Souris', 'prix' => 25),
    array('nom' => 'Écran', 'prix' => 300)
);

// Fonction générique de filtrage qui délègue la condition au callback
function filtrerPanier(array $produits, callable $condition): array {
    $resultat = array();
    foreach ($produits as $produit) {
        // Si le callback renvoie true, on garde le produit
        if ($condition($produit)) {
            $resultat() = $produit; // Rappel : syntaxe array() vue
précédemment
        }
    }
    return $resultat;
}

// Application : On veut uniquement les produits de plus de 50€
$produitsChers = filtrerPanier($panier, fn($p) => $p['prix'] > 50);

print_r($produitsChers); // Contient uniquement le Clavier et l'Écran
```